ISRM Lab 3: Security Testing Phase

Course: Information Security & Risk Management (ISRM)
Focus: Static Testing using Bandit & Dynamic Testing using OWASP ZAP

LAB OBJECTIVE

The objective of this lab is to perform both:

- **Static Application Security Testing (SAST)** using Bandit
- **Dynamic Application Security Testing (DAST)** using OWASP ZAP

on the Student Management System developed in previous labs.

This phase focuses on:

1. Testing the vulnerable branch
2. Testing the fixed branch
3. Generating security reports
4. Comparing vulnerable vs fixed results
5. Validating remediation effectiveness

PART A: GITHUB BRANCH STRUCTURE

Branch Name	Purpose
vulnerable-version	Contains insecure implementation for testing
fixed-version	Contains remediated secure implementation

Branch Commands:

```
git checkout vulnerable-version

git checkout fixed-version
```

PART B: STATIC TESTING USING BANDIT (SAST)

1. Installing Bandit

```
# Activate virtual environment

venv\Scripts\activate

# Install Bandit

pip install bandit

# Verify installation

bandit --version
```

Expected Output:

```
bandit 1.7.x
Python 3.x
```

2. Running Scan on Vulnerable Version

```
git checkout vulnerable-version

mkdir reports

bandit -r .

bandit -r . -f json
-o reports/bandit_report_vulnerable.json

bandit -r . -f html
-o reports/bandit_report_vulnerable.html

bandit -r . -ll
```

3. Vulnerabilities Found by Bandit

#	Bandit ID	File	Severity	CWE	Issue
1	B608	database.py	HIGH	CWE-89	SQL Injection
2	B201	app.py	HIGH	CWE-94	Flask Debug Enabled
3	B104	app.py	MEDIUM	CWE-605	Bind All Interfaces

4	B105	config.py	LOW	CWE-259	Hardcoded Password
5	B607	app.py	LOW	CWE-78	Unsafe Execution Path

4. Sample Bandit Output

```
Issue: B608
Severity: HIGH
Description: SQL Injection

Issue: B201
Severity: HIGH
Description: Flask debug=True

Issue: B105
Severity: LOW
Description: Hardcoded Password
```

5. Vulnerable SQL Injection Code

```
query = f"SELECT␣*␣FROM␣users
WHERE␣username='{username}'
AND␣password='{password}'"

cursor.execute(query)
```

Issue:

- Dynamic query construction
- Authentication bypass possible
- Database manipulation possible

6. Fixed SQL Injection Code

```
cursor.execute(
"SELECT␣*␣FROM␣users
WHERE␣username=?
AND␣password=?",
(username,password)
)
```

Fix Applied:

- Parameterized queries implemented
- SQL Injection prevented

7. Running Scan on Fixed Version

```
git checkout fixed-version

bandit -r . -f json
-o reports/bandit_report_fixed.json

bandit -r . -f html
-o reports/bandit_report_fixed.html
```

Expected Output:

```
HIGH: 0
MEDIUM: 0
LOW: 0

No vulnerabilities detected
```

PART C: DYNAMIC TESTING USING OWASP ZAP (DAST)

1. Installing OWASP ZAP

Download Link:

```
https://www.zaproxy.org/download/
```

Installation Steps:

1. Download OWASP ZAP
2. Install the application
3. Launch OWASP ZAP
4. Select temporary session

2. Browser Proxy Configuration

Setting	Value
Proxy Host	localhost
Proxy Port	8080

Protocol	HTTP/HTTPS

3. Running Vulnerable Application

```
git checkout vulnerable-version

python app.py
```

Application URL:

```
http://localhost:5000
```

4. Running OWASP ZAP Scan

Steps Performed:

1. Open OWASP ZAP
2. Enter target URL
3. Start Spider Scan
4. Run Active Scan
5. Analyze alerts generated

5. Vulnerabilities Found using OWASP ZAP

Vulnerability	**Risk Level**	**Description**
SQL Injection	HIGH	Database manipulation possible
Cross Site Scripting (XSS)	HIGH	JavaScript execution possible
CSRF Vulnerability	HIGH	Unauthorized actions possible
Missing Security Headers	MEDIUM	Missing CSP and X-Frame options
Session Cookie Without HttpOnly	MEDIUM	Session theft possible
Directory Browsing	MEDIUM	Sensitive files exposed
Information Disclosure	LOW	Debug information exposed

6. SQL Injection Attack Demonstration

Payload Used:

```
admin' OR '1'='1
```

Affected URL:

```
http://localhost:5000/login
```

Result:

- Login bypass successful
- Unauthorized access obtained

7. Cross Site Scripting (XSS) Test

Payload Used:

```
<script>alert('XSS')</script>
```

Result:

- JavaScript executed successfully
- Application vulnerable to XSS

8. Missing Security Header Alert

```
Missing Header:
Content-Security-Policy

Risk:
HIGH

Impact:
Increased XSS attack surface
```

9. Running Scan on Fixed Version

```
git checkout fixed-version

python app.py
```

Expected OWASP ZAP Result:

```
HIGH Alerts: 0

MEDIUM Alerts: Reduced

LOW Alerts: Minimal
```

PART D: STATIC VS DYNAMIC TESTING

Feature	Bandit (SAST)	OWASP ZAP (DAST)
Testing Type	Static Testing	Dynamic Testing
Code Execution Required	No	Yes
Analyzes Source Code	Yes	No
Tests Running Application	No	Yes
Finds Runtime Issues	No	Yes
Finds SQL Injection	Yes	Yes
Finds XSS	Limited	Yes
Tool Used	Bandit	OWASP ZAP

PART E: COMPARATIVE ANALYSIS

1. Vulnerable vs Fixed Comparison

Issue	Vulnerable Version	Fixed Version
SQL Injection	Present	Resolved
XSS	Present	Resolved
Hardcoded Secrets	Present	Resolved
Debug Mode	Enabled	Disabled
Security Headers	Missing	Added
CSRF Protection	Missing	Implemented

2. Severity Distribution

```
VULNERABLE VERSION

HIGH : 7
MEDIUM : 3
LOW : 2

TOTAL : 12 Findings
```

```
FIXED VERSION

HIGH : 0
MEDIUM : 1
LOW : 0

TOTAL : 1 Finding
```

PART F: OWASP TOP 10 MAPPING

Vulnerability	OWASP Category	CWE
SQL Injection	A03:2021 Injection	CWE-89
Cross Site Scripting	A03:2021 Injection	CWE-79
Hardcoded Credentials	A07:2021 Authentication Failures	CWE-259
CSRF	A01:2021 Broken Access Control	CWE-352
Debug Mode Exposure	A05:2021 Security Misconfiguration	CWE-94
Session Hijacking	A07:2021 Authentication Failures	CWE-614

PART G: GENERATED REPORTS

- bandit_report_vulnerable.json
- bandit_report_vulnerable.html
- bandit_report_fixed.json
- bandit_report_fixed.html
- owasp_zap_report.html

PART H: IMPLEMENTATION SUMMARY

Feature	Tool Used	Purpose
Static Testing	Bandit	Source Code Security Analysis
Dynamic Testing	OWASP ZAP	Runtime Vulnerability Analysis
Version Control	GitHub	Branch Management

Report Generation	JSON + HTML	Security Documentation
OWASP Mapping	OWASP Top 10	Risk Classification

CONCLUSION

In Lab 3, both Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) were successfully performed on the Student Management System.

Bandit was used for source code level security analysis, while OWASP ZAP was used to test the live running application for runtime vulnerabilities such as SQL Injection, XSS, CSRF, and security misconfigurations.

The vulnerable branch showed multiple HIGH severity findings, whereas the fixed branch successfully mitigated most of the vulnerabilities using secure coding practices and configuration hardening.

This lab demonstrated complete security testing lifecycle including vulnerability identification, exploitation, remediation validation, and comparative security analysis.

— End of Report —